



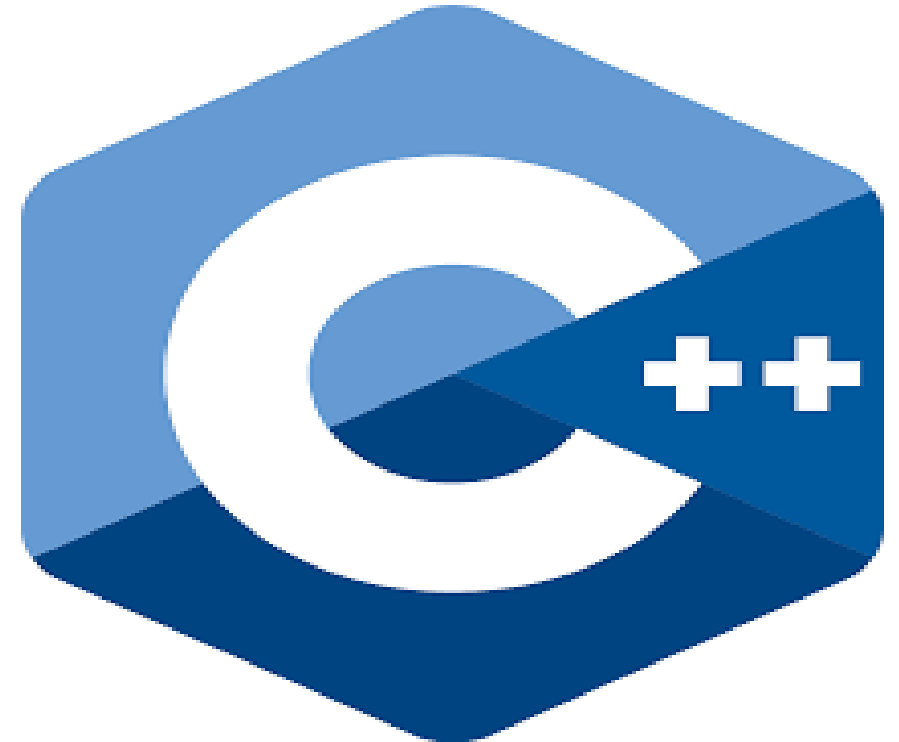
Marwadi
University
Marwadi Chandarana Group





Object Oriented Design and Programming (01AI0403)

Unit - 1 Introduction



- Introduction to Object Oriented Programming
- Procedural Vs. Object Oriented Programming
- Principles of OOP
- Access Modifiers
- Basics of a Typical C++ Environment
- Pre-processors Directives
- C++ Program structure
- Header Files and Namespaces
- Library files



- C++ was developed by **Dr. Bjarne Stroustrup**, as an extension to the C language.
- Danish computer scientist Bjarne Stroustrup began developing the new programming language shortly after joining the technical staff at **Bell Laboratories in 1979**.
- C++ gives programmers **a high level of control over system resources** and **memory**. It is one of the more-challenging programming languages to apply on a large scale.

Applications

YouTube



Microsoft Office



Spotify



Amazon.com



Google



Windows OS



Adobe Photoshop



Applications

Win Amp Media Player



Adobe Systems



MYSQL Server



Apple OS



Alias System



Google Applications



Bloomberg



Infosys Finacle



Thunderbird



Real-World Applications Of C++

- Games
- GUI-Based Applications
 - ✓ [Adobe Systems](#)
 - ✓ [Win Amp Media Player](#)
- Database Software
 - ✓ [MYSQL Server](#)
- Operating Systems
 - ✓ [Apple OS](#)
 - ✓ [Microsoft Windows OS](#)
- Browsers
 - ✓ [Mozilla Firefox](#)
 - ✓ [Thunderbird](#)
 - ✓ [Google Applications](#)
- Advanced Computation And Graphics
 - ✓ [Alias System](#)
- Banking Applications
 - ✓ [Infosys Finacle](#)

Real-World Applications Of C++

- Cloud/Distributed System
 - ✓ **Bloomberg**
- Compilers
- Embedded Systems
- Enterprise Software
- Libraries

- Programmers write instructions in various programming languages to perform their computation tasks.
- Programming languages allow programmers to code software.
- The three major families of languages are:
 - ✓ Machine level Language
 - ✓ Assembly level Language
 - ✓ High level Language

Machine level Language :

- Machine language is the language which a CPU understands
- It consists only of numbers: 0 & 1.
- Example of code:

1110100010101 111010101110

10111010110100 10100011110111

- Any computer can directly understand only its own machine language, defined by its hardware design.
- Machine dependent
- Difficult to program —**one misplaced 1 or 0 will cause the program to fail.**

Assembly level Language :

- Assembly languages are a step towards easier programming.
- Similar to machine language, but use naturally understandable symbols called “**Mnemonics**” for numeric instructions present in the machine language.
- Example:

Load 5, Load 1, Add

ADD 1001010, 1011010

- It works directly with microprocessor.
- Assembly language code needs to be **translated to machine language** before the computer processes it.
- Assembly language is **intermediate** between high level language and machine code.

High level Language :

- High-level languages represent a giant leap towards easier programming.
- The syntax of HL languages is similar to English.
- The high-level programming languages are broadly categorized in to two categories:
 - ✓ Procedural oriented programming languages (POP)
Ex: C language
 - ✓ Object-Oriented Programming languages (OOP)
Ex. C++, Java, C#

Procedure Oriented Programming Language

- In the procedure oriented approach, the problem is viewed as **sequence of things** to be done such as reading , calculation and printing.
- Procedure oriented programming basically consist of writing a list of instruction or actions for the computer to follow and **organizing these instruction into groups** known as functions.

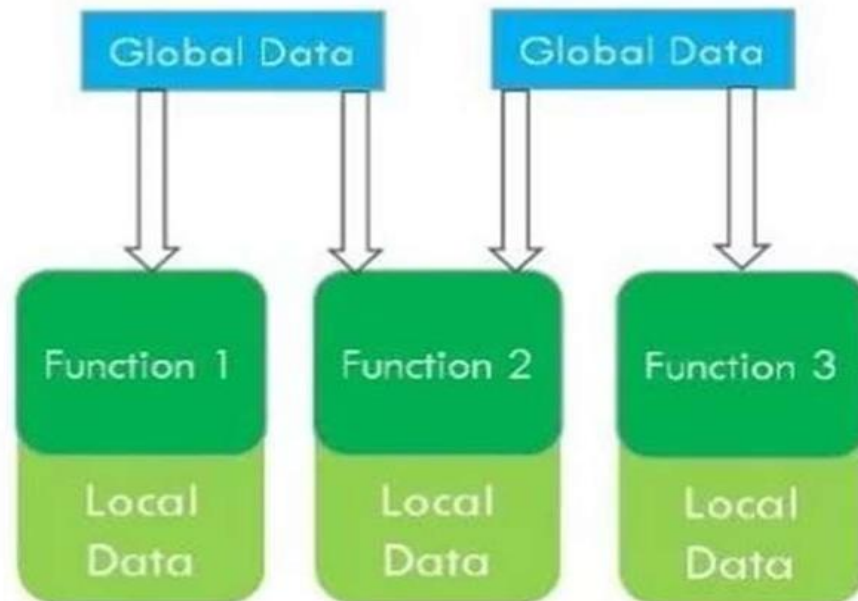
Characteristics of procedure oriented programming:

1. Emphasis is on doing things(algorithm).
2. Large programs are divided into smaller programs known as functions.
3. Most of the functions share global data.
4. Data move openly around the system from function to function.
5. Function transforms data from one form to another.
6. Employs top-down approach in program design.

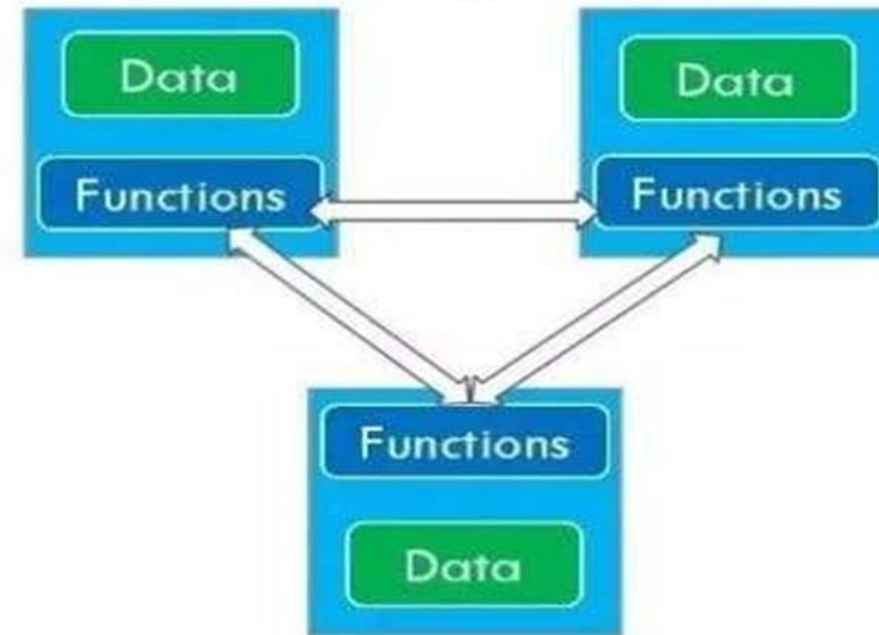
Procedural Vs. Object Oriented Programming



Procedural Oriented Programming



Object Oriented Programming



The **disadvantage** of the procedure oriented programming languages is:

1. Global data access
2. It does not model real word problem very well
3. No data hiding

Object Oriented Programming

- Object-oriented programming (OOP) is a programming paradigm that emphasizes the concept of objects. An object is a real-world entity that has state and behavior.
- In OOP, a program is designed by creating classes that represent objects, and these classes encapsulate the data and behavior of those objects.
- OOP provides a way to organize complex programs by breaking them down into smaller, more manageable pieces. It also allows for code reusability and modularity, making it easier to maintain and extend code over time.

Benefits of OOPs:

OOP offers several benefits to both the program designer and user.

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code on other parts of the program.
- It is possible to have multiple instances of an object to co-exist without any interference.
- It is possible to map objects in the problem domain to those objects in the program.

Benefits of OOPs: (Continued...)

- It is easy to partition the work in a project based on objects.
- Object-oriented systems can be easily upgraded from small to large systems.
- Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- Software complexity can be easily managed.

Procedural Vs. Object Oriented Programming



Marwadi
University
Marwadi Chandarana Group



Procedural Vs. Object Oriented Programming



Parameters	POP	OOP
Basic Definition	POP is structure or procedure-oriented.	OOP is object-oriented.
Program Division	The program is divided into functions.	The program is divided into objects.
Approach	Top-down approach	Bottom-Up approach
Data Control	Every function has different data, so there's no control over it.	Data in each object is controlled on its own.
Entity Linkage	Parts of a program are linked through parameter passing.	Object functions are linked through message passing.
Expansion	Expanding data and function is not easy.	Adding new data and functions is easy.
Inheritance	Inheritance is not supported.	Inheritance is supported in three modes: public, private & protected.
Access control	No access modifiers supported.	Access control is done with access modifiers.

Difference between POP and OOP



Parameters	POP	OOP
Basic Definition	POP is structure or procedure-oriented.	OOP is object-oriented.
Program Division	The program is divided into functions.	The program is divided into objects.
Approach	Top-down approach	Bottom-Up approach
Data Control	Every function has different data, so there's no control over it.	Data in each object is controlled on its own.
Entity Linkage	Parts of a program are linked through parameter passing.	Object functions are linked through message passing.
Expansion	Expanding data and function is not easy.	Adding new data and functions is easy.
Inheritance	Inheritance is not supported.	Inheritance is supported in three modes: public, private & protected.
Access control	No access modifiers supported.	Access control is done with access modifiers.

Difference between POP and OOP



Example



Company Name: Hyundai

Model Name: Elite i20 Fuel

Type: Petrol / Diesel

Mileage: 12 km/ltr

Price: 8 lac

POP

```
Void main( )  
{  
    char company_nm[20]; char  
    model_nm[20]; char  
    fuel_type[10]; float mileage;  
    double price;  
  
    //other code operation  
}
```

OOP

```
Class Car  
{  
    char company_nm[20];  
    char model_nm[20];  
    char fuel_type[10];  
    float mileage;  
    double price;  
    //other code operation  
}  
  
Void main()  
{  
    Car Object1; Car  
    Object2;  
}
```

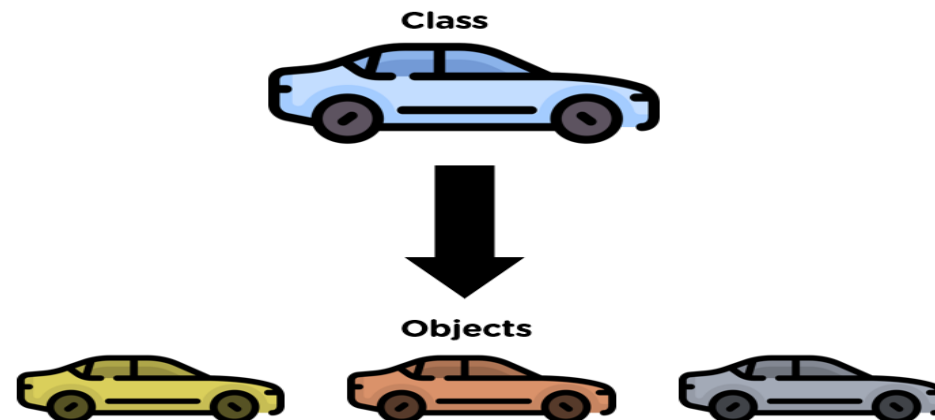
There are some basic concepts that act as the building blocks of OOPs.

- Class
- Object
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic binding
- Message Passing

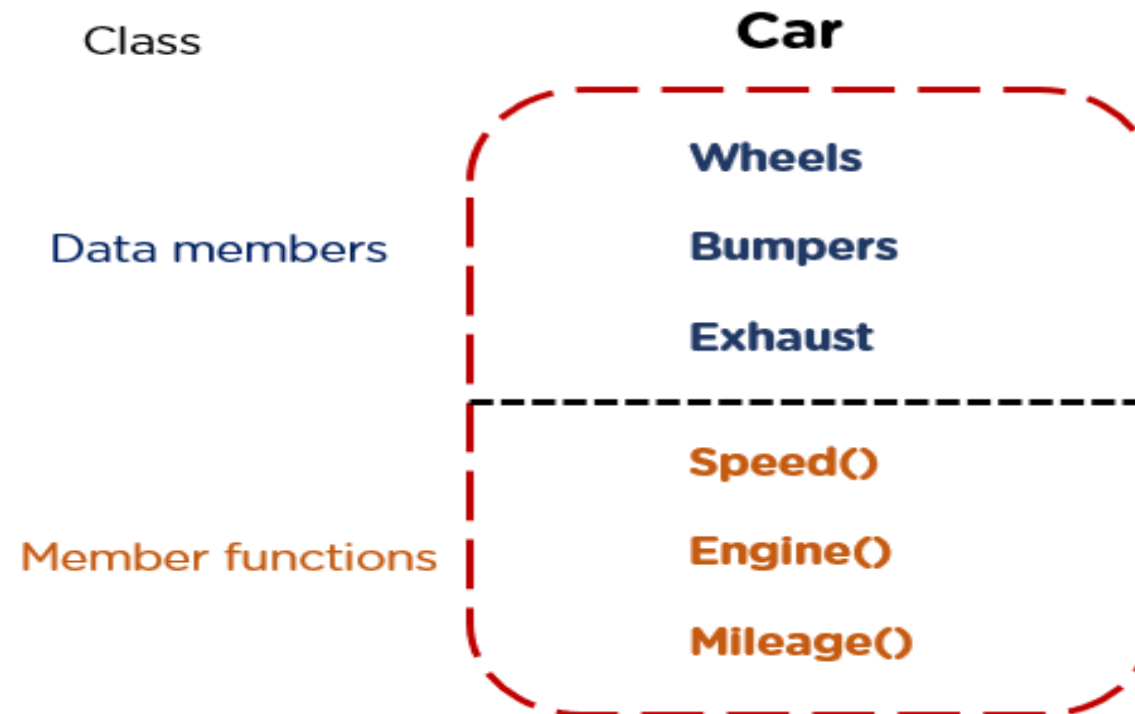
Class:

- Class is Blueprint for any entity which defines properties and function.
- It is a user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class.
- Example:

Class Name - CAR

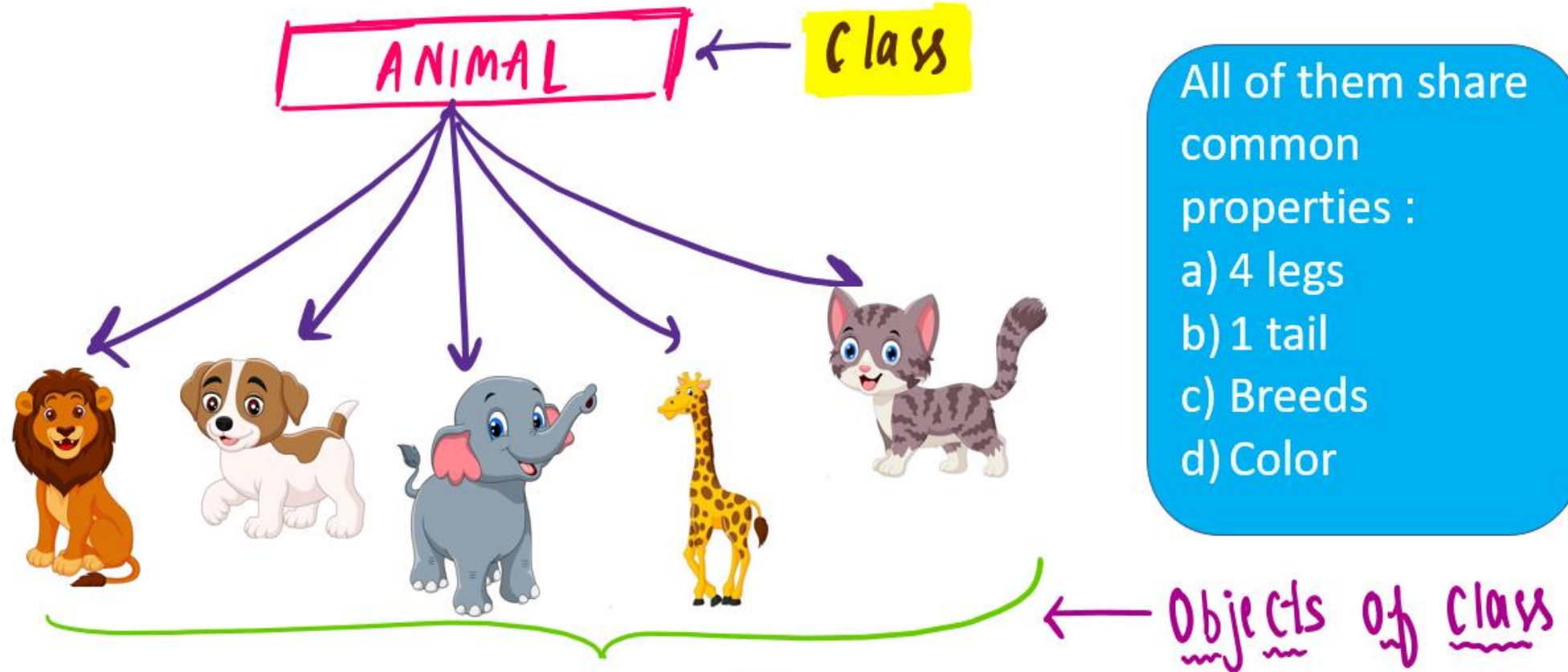


- A class contains data members (variables) and member functions(methods). These member functions are used to manipulate the data members inside the class.

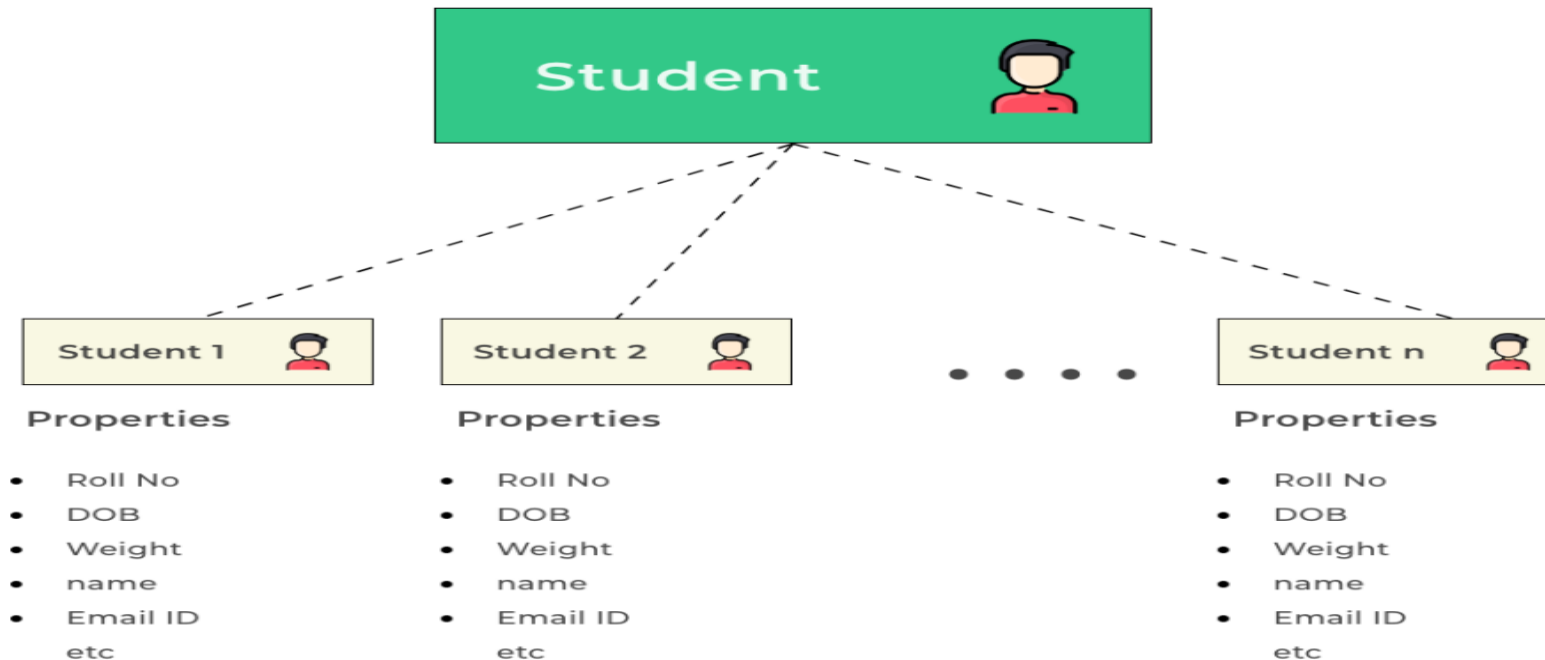


- **Object** is real world entity or it is instance of Class.
- It may be represent a person, a place, a bank account, any things, a table of data or any item that the program must handle.
- When a class is defined, only the specification for the object is defined, **no memory or storage is allocated**.
- To use the data and access functions defined in the class, we need to create objects.

Class and objects - Example



Classes & Objects in C++



Classes are an interface to create multiple real-life objects(instances) that share common properties.

- It is a process of hiding irrelevant details from the user.
- Abstraction/Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.



Using Abstraction



Without Abstraction

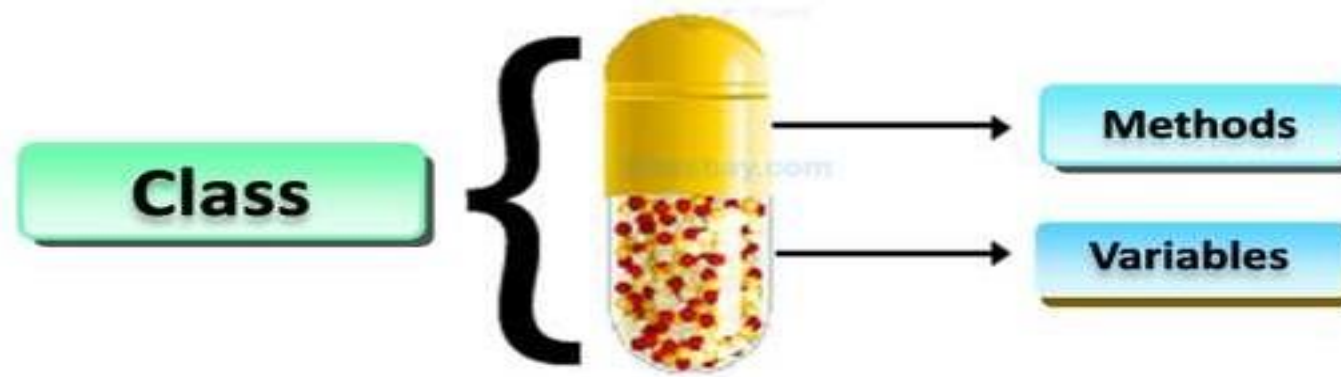
Example - 1

Consider a real life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of car or applying brakes will stop the car but he does not know about how on pressing accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of accelerator, brakes etc in the car. This is what abstraction is.

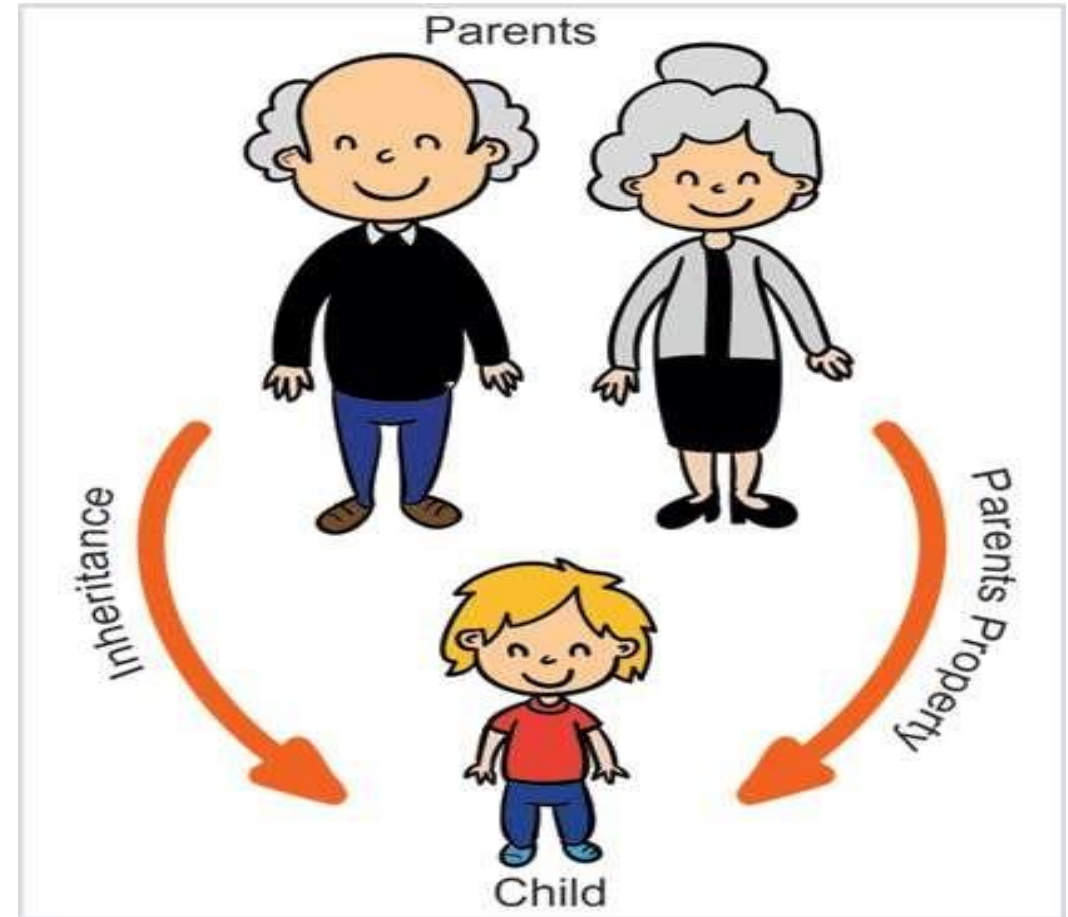
Example - 2

The calculator shown in the figure has to be powered by a battery source. How the battery module works for the calculator is not necessary to know for the user who uses calculator.

- The word “**encapsulate**” refers to the act of enclosing something, i.e Keeping the data and function into a single unit like a capsule.
- Encapsulation is a process of combining **data members** and **member functions** in a single unit called class.
- As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.



- The capability of a class to **derive properties and characteristics** from another class is called **Inheritance**.
- Inheritance is one of the most important feature of Object Oriented Programming.
- There are two types of classes. They are,
 - ✓ Child class
 - ✓ Parent class



- **Child Class:**
The class that inherits properties from another class is called **sub class** or **derived class**.
- **Parent Class:**
The class whose properties are inherited by sub class is called **base class** or **super class**.

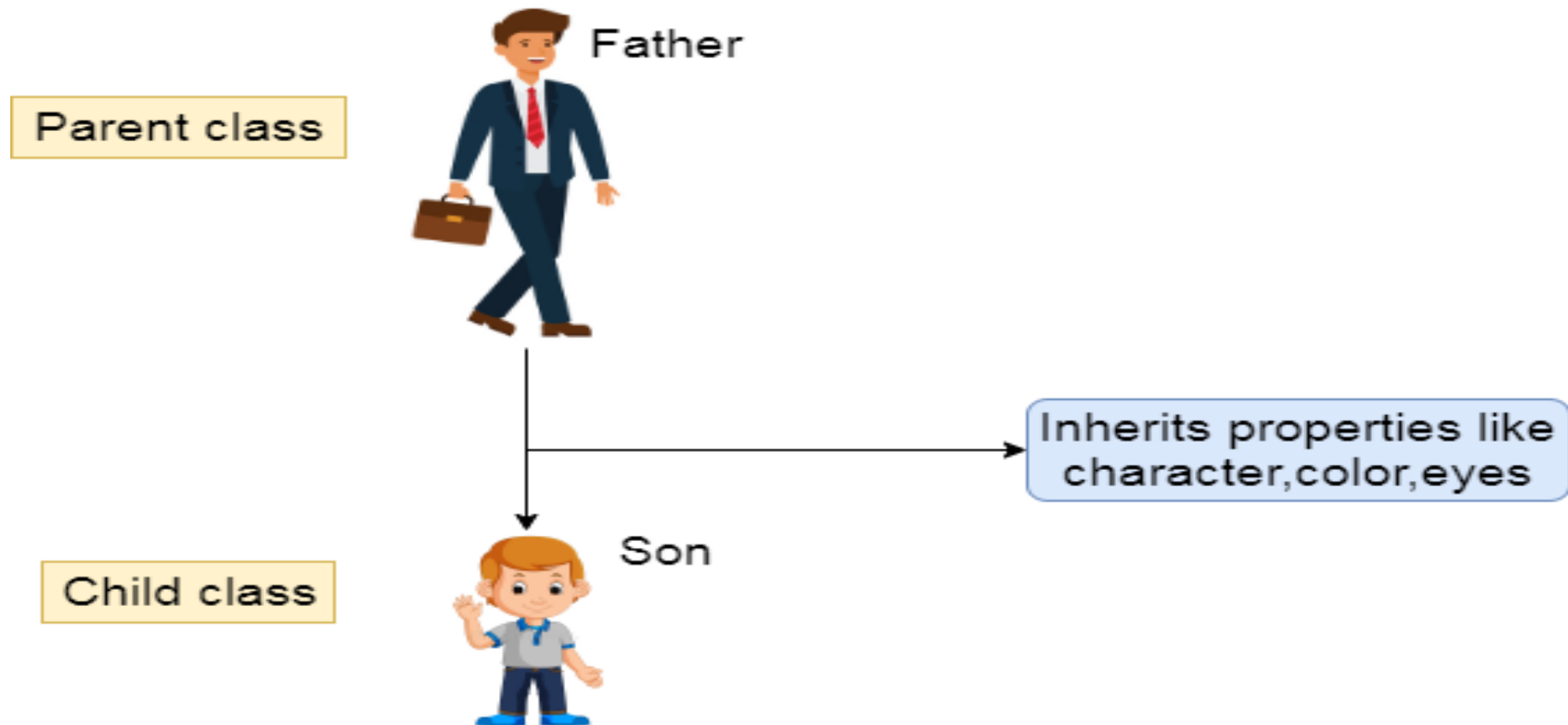
Real Time Example:

An insect could be represented by an Insect superclass in the animal world. Insects all have the same characteristics, such as six legs and an exoskeleton. For grasshoppers and ants, subclasses could be created. They inherently share all bug features because they inherit or are derived from the Insect class.

Inheritance - Example



Marwadi
University
Marwadi Chandarana Group



- We can define polymorphism as the ability of a message to be displayed in **more than one form.**
- Polymorphism in OOPs is inseparable and an essential concept of every object-oriented programming language.
- An object or reference basically can take multiple forms in different instances.
- As the word suggests, ‘**poly**’ means ‘**many**’ and ‘**morph**’ points at ‘**forms**’; thus, polymorphism as a whole would mean ‘a **property of having many forms.**’

Polymorphism - Example



Marwadi
University
Marwadi Chandarana Group



- Person may be different form in different places.

In Shopping malls behave like

CUSTOMER

In Bus behave like

PASSENGER

In School behave like

STUDENT

At Home behave like

SON



- Polymorphism in C++ is broadly classified into two categories based on their characteristic features.
 - ✓ Compile-time Polymorphism
 - ✓ Run-time Polymorphism

Compile-time Polymorphism

- When the relationship between the definition of different functions and their function calls, is determined during the **compile-time**, it is known as compile-time polymorphism.
- This type of polymorphism is also known as **static or early binding** polymorphism.
- All the methods of compile-time polymorphism **get called or invoked** during the compile time.
- The implementation of compile-time polymorphism is achieved in **two ways**:
 - ✓ **Function overloading**
 - ✓ **Operator overloading**

Runtime Polymorphism

- In runtime polymorphism, the compiler resolves the object at **run time** and then it decides which function call should be associated with that object.
- It is also known as **dynamic or late binding** polymorphism. This type of polymorphism is executed through virtual functions and function overriding.
- All the methods of runtime polymorphism **get invoked during the run time**.
- The implementation of run time polymorphism can be achieved in **two ways**:
 - ✓ **Function overriding**
 - ✓ **Virtual functions**

- **Binding** refers to the linking of a procedure call to the code to be executed in response to the call.
- Dynamic Binding (also known as late binding) means that the **code associated with a given procedure call** is not known until the time of the call at run-time.
- It is associated with **polymorphism and inheritance**.
- A function call associated with a polymorphic reference depends on the dynamic type of that reference.

- An object-oriented program consists of a **set of objects that communicate with each other.**
- A message is a form of request that is made to an object to perform a function specific to the object to which the request was made.
- A message is a form of communication between objects that make an object perform a task for the system.
- Message passing in C++ is the method of communication between two or more objects.
- A message is used to communicate with other objects by invoking the function of other objects.
- A real-life entity modeled using the oops concept in C++ provides an example of message passing.

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code on other parts of the program.
- It is possible to have multiple instances of an object to co-exist without any interference.
- It is possible to map objects in the problem domain to those objects in the program.



- It is easy to partition the work in a project based on objects.
- Object-oriented systems can be easily upgraded from small to large systems.
- Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- Software complexity can be easily managed.

- **Data hiding** is an important concept of Object-Oriented Programming, implemented with these 'Access modifiers' help. It is also known as **Access Specifier**.
- Access Specifiers in a class decide the accessibility of the class members, like variables or methods in other classes.
- That is, it will decide whether the members or methods will get directly accessed by the blocks present outside the class or not, depending on the type of Access Specifier.

There are 3 types of access modifiers available in C++.

Private:

- A private member variable or function cannot be accessed, or even viewed from outside the class. Only the class and friend functions can access private members.

Protected:

- A protected member variable or function is very similar to a private member but it provided one additional benefit that they can be accessed in child classes which are called derived classes.

Public:

- A public member is accessible from anywhere outside the class but within a program.

Syntax of Declaring Access Specifiers

```
class ClassName
{
private:
// Declare private members/methods here.

public:
// Declare public members/methods here.

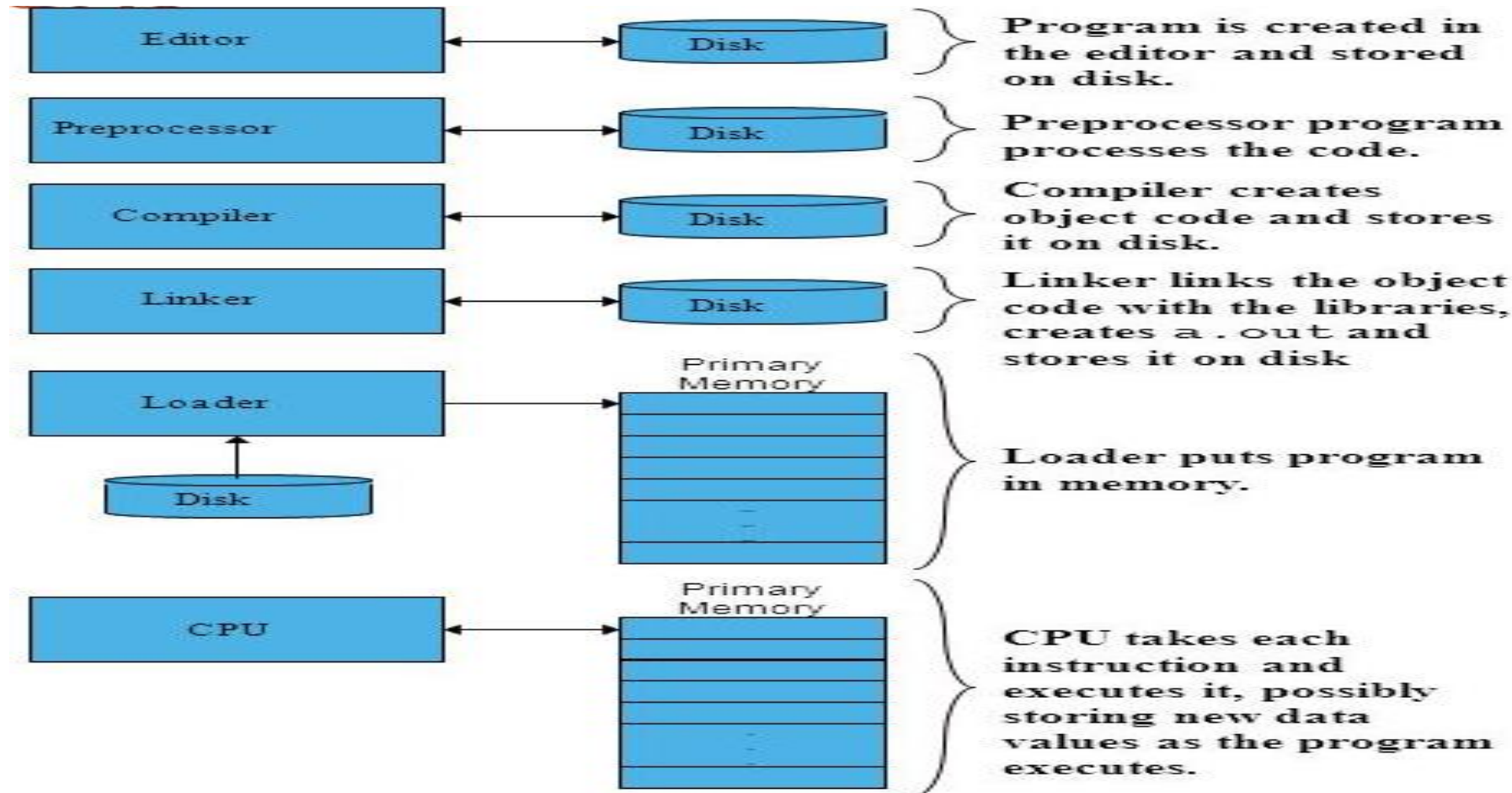
protected:
// Declare protected members/methods here.
};
```

Specifiers	within same class	in derived class	outside the class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

```
#include <iostream>
using namespace std;
class access
{
private:
    int a_pri = 10;
protected:
    int b_pro = 20;
public:
    int c_public = 30;
};
```

- C++ systems generally consist of three parts: a program development environment, the language and the C++ Standard Library.
- C++ programs typically go through six phases:
 - ✓ Editor
 - ✓ Preprocessor
 - ✓ Compilers
 - ✓ Linkers
 - ✓ Loaders
 - ✓ Execute

Basics of a Typical C++ Environment



Phase 1: Creating a Program

- It consists of editing a file with an editor program C++ source code filenames often end with the **.cpp extension**.
- In Windows: Notepad++, turbo C++, Code blocks, Visual Studio etc.

Phases 2 and 3: Preprocessing and Compiling a C++ Program

- A preprocessor program executes automatically before the compiler's translation phase begins.
- The C++ preprocessor obeys commands called preprocessor directives, which indicate that certain manipulations are to be performed on the program before compilation.

Phase 4: Linking

- A linker links the object code with the code for the missing functions to produce an executable program.

Phase 5 : Loading

- Before a program can be executed, it must first be placed in memory. This is done by the loader, which takes the executable program from disk and transfers it to memory.
- Additional components from shared libraries that support the program are also loaded.

Phase 6 : Execution

- Finally, the computer, under the control of its CPU, executes the program one instruction at a time.

- The preprocessor is a **program that is invoked by the compiler** to process code before compilation.
- Commands for that program, known as directives, are lines of the source file beginning with the **character #**, which distinguishes them from lines of source program text.
- The effect of each preprocessor directive is a change to the text of the source code, and the result is a new source code file, which does not contain the directives.
- The preprocessed source code, an intermediate file, must be a valid C or C++ program, because it becomes the input to the compiler.

- There are 4 Main Types of Preprocessor Directives:
 - ✓ Macros
 - ✓ File Inclusion
 - ✓ Conditional Compilation
 - ✓ Other directives

Macros

- Macros are pieces of code in a program that is given some name.
- Whenever this name is encountered by the compiler, the compiler replaces the name with the actual piece of code.
- The '**#define**' directive is used to define a macro.

File Inclusion

- This type of preprocessor directive tells the compiler to include a file in the source code program.
- There are two types of files that can be included by the user in the program:
 - 1) Header files or Standard files:
 - These files contain definitions of pre-defined functions like `cout()`, `cin()`, etc.
 - These files must be included to work with these functions. Different functions are declared in different header files.

Syntax:

```
#include< file_name >
```

2) User-defined files:

When a program becomes very large, it is a good practice to divide it into smaller files and include them whenever needed. These types of files are user-defined files.

Syntax:

```
#include "filename"
```

Documentation Section
Header Files Declaration Section
Preprocessor Statements
Global Declaration
Class Definition
Main Function <pre>{ // Main method definition }</pre>
User Defined Function

Documentation Section:

- This section comes first and is used to document the logic of the program that the programmer going to code.
- It can be also used to write for purpose of the program.
- Whatever written in the documentation section is the comment and is not compiled by the compiler.
- Documentation Section is optional since the program can execute without them.

Documentation Section: (Contin..)

- Comment lines are used to create the documentation.
 - ✓ Multiple line comment - `/*.....*/`
 - ✓ Single line comment - `//`

Example:

- `/* This is a C++ program to find the factorial of a number. The basic requirement for writing this program is to have knowledge of loops*/`
- `// Factorial program`

Header Files:

- Generally, a program includes various programming elements like built-in functions, classes, keywords, constants, operators, etc. that are already defined in the standard C++ library.
- In order to use such pre-defined elements in a program, an appropriate header must be included in the program.
- Standard headers are specified in a program through the preprocessor directive `#include`.
- When the compiler processes the instruction `#include<iostream>`, it includes the contents of the stream in the program. This enables the programmer to use standard input, output, and error facilities that are provided only through the standard streams defined in `<iostream>`.

Namespaces:

- A namespace permits grouping of various entities like classes, objects, functions, and various C++ tokens, etc. under a single name.
- Any user can create separate namespaces of its own and can use them in any other program.
- In the below snippets, namespace std contains declarations for cout, cin, endl, etc. statements.

```
using namespace std;
```

Definition Section:

- It is used to declare some constants and assign them some value.
- In this section, anyone can define your own datatype using primitive data types.
- In **#define** is a compiler directive which tells the compiler whenever the message is found to replace it with “Factorial\n”.
- **typedef int INTEGER;** this statement tells the compiler that whenever you will encounter INTEGER replace it by int and as you have declared INTEGER as datatype you cannot use it as an identifier.

Global Declaration Section:

- Here, the variables and the class definitions which are going to be used in the program are declared to make them global.
- The scope of the variable declared in this section lasts until the entire program terminates.
- These variables are accessible within the user-defined functions also.

Function Declaration Section:

- It contains all the functions which our main functions need.
- Usually, this section contains the User-defined functions.
- This part of the program can be written after the main function but for this, write the function prototype in this section for the function which for you are going to write code after the main function.

Main Function:

- The main function tells the compiler **where to start the execution** of the program.
- The execution of the program **starts with the main function**.
- **All the statements** that are to be executed are written in the main function.
- The compiler executes all the instructions which are written in the curly **braces {} which encloses** the body of the main function.
- Once all instructions from the main function are executed, control comes out of the main function and the program terminates and no further execution occur.

Headers	Description
iostream	This header contains the prototype for standard input and output functions used in C++ like cin, cout, etc.
cmath	This is the header containing various math library functions.
iomanip	This header contains stream manipulator functions that allow us to format the stream of data.
cstdlib	The header cstdlib contains various functions related to conversion between text and numbers, memory allocation, random numbers, and other utility functions.
ctime	ctime contains function prototypes related to date and time manipulations in C++.
cctype	This header includes function prototypes that test the type of characters (digit, punctuation, etc.). It also has prototypes that are used to convert from uppercase to lowercase and another way around.

Headers	Description
cstring	cstring header includes function prototypes for C-style string-processing functions.
cstdio	This header contains function prototypes for the C-style standard input/output library functions which we included initially in stdio.h
fstream	Function prototypes for functions that perform input/output from/to files on disk are included infstream header.
climits	climits header has the integral size limits of the system.
cassert	cassert header contains macros and variables for adding diagnostics that help us in program debugging.
cfloat	This header file contains the size limits for floating-point numbers on the system.
string	The header string defines the class string of the C++ Standard Library.

THANK YOU

